Atelier 4 : Express et une base de données

Vous reprenez l'API GreenCart de l'atelier 3, telle que son corrigé l'a laissée. Mêmes fichiers, mêmes middlewares (journal, validation, erreurs), mêmes réponses d'erreur `{ "message": "..." }`. Ce qui change : les produits ne vivent plus dans un tableau en mémoire, mais dans une vraie base de données. Le modèle en mémoire de l'atelier 3 laisse sa place à un dépôt, `depot/`, qui parle à la base. Un produit créé reste en place quand le serveur redémarre.

> **Les numéros de slides** sont ceux du diaporama du cours Node.js et Express. Dans la version PDF, la slide 12 est masquée : à partir de la slide 13, la page du PDF vaut le numéro de slide moins 1. La slide 22 est la page 21 du PDF.

Les exercices sont classés par priorité. Faites-les dans l'ordre. Les exercices **indispensables** d'abord (1 et 2), puis les **souhaitables** (3 et 4), puis le **bonus** si vous avez le temps. Les exercices 3 et 4 ne dépendent pas l'un de l'autre : vous pouvez passer directement à la commande.

1. Choisir sa version

Le même exercice existe en deux versions. Les routes, les exercices et le vérificateur sont identiques.

Dossier	Base	Pour qui
`mongo/`	MongoDB avec Mongoose, comme dans le cours	Webecom
`mysql/`	MySQL avec `mysql2`, en SQL	DWWM et CDA

Si votre formateur vous indique une autre version, suivez son choix. Vous travaillez ensuite **dans un seul dossier**, celui de votre version.

2. Démarrer

Il vous faut Node 18 ou plus (`node -v`) et Docker Desktop, ouvert.

```
cd mongo            # ou : cd mysql

# 1. La base, dans un conteneur Docker. Les données GreenCart sont chargées au premier
lancement.
docker compose up -d

# 2. Les réglages. Le fichier .env n'est jamais rendu ni publié.
cp .env.example .env

# 3. Les dépendances, puis le serveur
npm install
npm run dev         # nodemon relance le serveur à chaque modification

# 4. Dans un second terminal, depuis le même dossier
node ../verifier.js
```

Sous Windows, dans l'invite de commandes, `cp` s'écrit `copy`. PowerShell accepte `cp`.

Le serveur affiche `Base MongoDB connectée.` ou `Base MySQL connectée.`. MySQL met environ 30 secondes à démarrer la première fois. Le serveur réessaie tout seul.

Ouvrez http://localhost:3000/api/sante dans le navigateur. Vous devez lire `"connectee": true`.

Le vérificateur affiche une ligne par exercice : `OK`, ou `À revoir` avec une explication. Relancez-le après chaque correction, autant de fois que vous voulez. Il crée des produits de test nommés `[Vérif] ...`. C'est normal.

Repartir de zéro : `docker compose down -v` efface la base, puis `docker compose up -d` la recrée avec les données d'origine.

3. Ce qui est déjà écrit

```
mongo/ ou mysql/
├── docker-compose.yml     la base, sur le port 27118 (Mongo) ou 3318 (MySQL)
├── init/                  le chargement des données GreenCart
├── server.js, app.js      comme dans le cours (slide 14)
├── config/db.js           la connexion à la base : déjà écrite, n'y touchez pas
├── models/                (Mongo seulement) les schémas et modèles Mongoose
├── routes/                produits, commandes, auth, santé
├── controllers/           produit.controller.js est complet
├── middlewares/           journal, validation, erreurs : ceux de l'atelier 3. auth :
squelette pour le bonus
└── depot/                 l'accès aux données : c'est là que vous écrivez
```

Ce qui est repris de l'atelier 3. `app.js`, `server.js`, `routes/produit.routes.js` et les trois middlewares sont ceux de son corrigé. Le journal affiche toujours `Requête reçue : GET /api/produits`. La validation applique les mêmes règles au produit : nom, catégorie, prix, stock. Le middleware d'erreur répond toujours `{ "message": "..." }`. Il a une seule ligne de plus : il demande à `config/db.js` de traduire les erreurs de la base. `controllers/produit.controller.js` garde ses cinq fonctions : `listerProduits`, `afficherProduit`, `creerProduit`, `modifierProduit`, `supprimerProduit`. Elles sont devenues `async`, car une base répond avec un délai : chaque appel au dépôt prend un `await`.

Le dépôt (`depot/`) est la seule partie qui parle à la base. Il remplace `models/produit.model.js` de l'atelier 3 et garde ses noms : `trouverTous`, `trouverParId`, `creer`, `remplacer`, `supprimer`. Le contrôleur appelle `depot.trouverTous()`, `depot.trouverParId(id)`... sans savoir si la base est MongoDB ou MySQL. Une fonction pas encore écrite répond `501` avec son nom. Le vérificateur vous le dira.

Les règles que la validation de l'atelier 3 ne connaît pas, l'unité et le producteur, c'est la base qui les impose. `champsProduit(donnees)`, déjà écrite en haut de `depot/produits.js`, garde les champs d'un produit comme le faisait le modèle de l'atelier 3. Un produit sans unité est alors refusé par la base, et l'API répond `400`.

Le cours montre le chemin d'une requête : route, middleware, contrôleur, modèle, base (slide 40). Ici, le dépôt prend la place du modèle.

4. Les données GreenCart

Un produit, tel que l'API le renvoie :

```
{ "id": 4, "nom": "Carottes", "categorie": "legumes", "prix": 1.8, "unite": "kg", "stock":
60, "bio": true, "producteurId": 1 }
```

Avec MongoDB, `id` et `producteurId` sont des textes comme `"6730c1f2a1b2c3d4e5f60718"`. Avec MySQL, ce sont des nombres.

- 4 producteurs : `id`, `nom`, `ville`.
- 12 produits. Catégories : `fruits`, `legumes`, `cremerie`, `epicerie`. Unités : `kg`, `piece`, `botte`, `pot`.
- Le prix est en euros, **hors taxes**.

Avec MySQL, les tables sont `producteurs`, `produits` (colonne `producteur_id`), `commandes`, `lignes_commande` et `utilisateurs`. Le schéma complet est dans `mysql/init/01-schema.sql`. Lisez-le avant de commencer.

Exercice 1 (indispensable) : lire les produits depuis la base

Ce qui est demandé. Dans `depot/produits.js`, écrivez `trouverTous(filtres)` et `trouverParId(id)`.

- `trouverTous` renvoie tous les produits. Si `filtres.categorie` est rempli, seulement ceux de cette catégorie.
- `trouverParId` renvoie le produit qui a cet id, ou `null` s'il n'existe pas.

À l'atelier 3, le contrôleur filtrait la catégorie avec une boucle `for`. Ici, c'est la base qui filtre : c'est son travail.

Rappel. Cours Node.js et Express, slides 34 à 39 (base de données, Mongoose, schéma, modèle, lien avec l'API REST).

- Mongo : le modèle `Produit` est dans `models/produit.model.js`. `Produit.find()` lit tout, `Produit.find({ categorie: 'fruits' })` filtre, `Produit.findById(id)` lit un document.
- MySQL : une requête **paramétrée**. Le `?` est remplacé par la valeur, que MySQL traite toujours comme une simple valeur, jamais comme du SQL.

```
// MySQL : lire des lignes. Le résultat est un tableau, on prend son premier élément.
const [lignes] = await pool.execute(`SELECT ${COLONNES} FROM produits WHERE id = ?`, [id]);
```

```
SELECT id, nom FROM produits WHERE categorie = ? ORDER BY id
```

Ne collez jamais la valeur du client dans le texte SQL (`"... WHERE categorie = '" + categorie + "'"`). Un visiteur pourrait y glisser son propre SQL : c'est une **injection SQL**. Le vérificateur essaie.

Résultat attendu. http://localhost:3000/api/produits renvoie les 12 produits. http://localhost:3000/api/produits?categorie=legumes renvoie Carottes, Poireaux, Pommes de terre et Radis. Un id inconnu donne `404` et `{ "message": "Produit introuvable" }`.

Indice. Dans `trouverTous`, commencez par `if (filtres.categorie)` : une requête avec le filtre, sinon une requête sans filtre. Sans paramètre, le contrôleur envoie `{ categorie: undefined }`. Chercher `undefined` ne renvoie rien avec Mongo, et une erreur avec MySQL. `trouverParId` doit renvoyer `null` quand rien n'est trouvé. Le contrôleur s'occupe déjà du 404. Mongo : commencez `trouverParId` par `if (!idValide(id)) return null;`. MySQL : `lignes` est vide quand rien n'est trouvé. Testez `lignes.length === 0`.

À rendre. `depot/produits.js`.

Exercice 2 (indispensable) : créer, modifier, supprimer un produit

Ce qui est demandé. Toujours dans `depot/produits.js`, écrivez `creer(donnees)`, `remplacer(id, donnees)` et `supprimer(id)`.

- `creer` renvoie le produit créé, **avec son id**.
- `remplacer` renvoie le produit **après** modification, ou `null` s'il n'existe pas.
- `donnees` est le corps de la requête. Passez-le d'abord dans `champsProduit(donnees)`.
- `supprimer` renvoie `true` si le produit existait, `false` sinon.

Le contrôleur et la validation sont déjà écrits. Les codes HTTP aussi : 201, 200, 204, 400, 404.

Rappel. Cours Node.js et Express, slide 19 (méthodes POST, PUT, DELETE), slide 20 (codes de statut), slide 39.

- Mongo : `Produit.create(donnees)`, `Produit.findByIdAndUpdate(id, donnees, { new: true })`, `Produit.findByIdAndDelete(id)`. L'option `{ new: true }` renvoie le document modifié, et non l'ancien.
- MySQL : `INSERT`, `UPDATE`, `DELETE`, toujours avec des `?`.

```
INSERT INTO produits (nom, categorie, prix, unite, stock, bio, producteur_id) VALUES
(?, ?, ?, ?, ?, ?, ?)
UPDATE produits SET nom = ?, prix = ? WHERE id = ?
DELETE FROM produits WHERE id = ?
```

Après un `INSERT`, `resultat.insertId` contient l'id donné par MySQL. Après un `UPDATE` ou un `DELETE`, `resultat.affectedRows` compte les lignes touchées.

```
const [resultat] = await pool.execute('DELETE FROM produits WHERE id = ?', [id]);
```

Résultat attendu.

```
curl -i -X POST http://localhost:3000/api/produits \
  -H "Content-Type: application/json" \
  -d '{"nom":"Confiture de
figues","categorie":"epicerie","prix":5.2,"unite":"pot","stock":20,"bio":false,"producteurId":
<id du producteur>}'
```

Réponse `201 Created` avec le produit et son id. Arrêtez puis relancez le serveur : le produit est toujours là. Vous pouvez aussi tester avec Postman ou Thunder Client. Prenez un `producteurId` existant dans la réponse de `GET /api/produits`. Avec Mongo, mettez-le entre guillemets.

Indice. MySQL : pour renvoyer le produit créé, réutilisez votre fonction `trouverParId(resultat.insertId)`. Mongo : avant d'enregistrer, appelez `verifierProducteur(produit.producteurId)`, déjà écrite. MongoDB n'a pas de clé étrangère. Si `affectedRows` vaut 0, aucun produit n'a cet id : renvoyez `null` ou `false`.

À rendre. `depot/produits.js`.

Exercice 3 (souhaitable) : les producteurs

Ce qui est demandé. Ajoutez la ressource producteurs, sur le modèle exact des produits.

Méthode et chemin	Réponse
`GET /api/producteurs`	200, tous les producteurs
`GET /api/producteurs/:id`	200, ou 404
`GET /api/producteurs/:id/produits`	200, les produits de ce producteur, ou 404 si le producteur n'existe pas
`POST /api/producteurs`	201, ou 400 si `nom` ou `ville` manque
`PUT /api/producteurs/:id`	200, ou 400, ou 404
`DELETE /api/producteurs/:id`	204, ou 404

Quatre fichiers à écrire ou à créer, chacun en copiant celui des produits :

1. `depot/producteurs.js` : les fonctions sont annoncées (`trouverTous`, `trouverParId`, `creer`, `remplacer`, `supprimer`, `listerProduits`), à vous de les écrire.
2. `middlewares/validation-producteur.middleware.js` : à créer, en copiant `validation.middleware.js`. `validerProducteur` refuse un nom ou une ville vide.
3. `controllers/producteur.controller.js` : à créer, en copiant `produit.controller.js`.
4. `routes/producteur.routes.js` : à créer, puis à brancher dans `app.js` (deux lignes à décommenter).

Rappel. Cours Node.js et Express, slide 22 (structure du projet), slides 23 à 25 (une ressource, un fichier de routes), slides 26 à 29 (middlewares), slide 39.

- Mongo : les produits d'un producteur sont ceux dont le champ `producteurId` vaut son id : `Produit.find({ producteurId: id })`.
- MySQL : la colonne `producteur_id` de `produits` est une **clé étrangère** vers `producteurs.id`.

```
SELECT id, nom FROM produits WHERE producteur_id = ?
-- avec une jointure, pour avoir aussi le nom du producteur :
SELECT p.nom, f.nom AS producteur FROM produits p JOIN producteurs f ON f.id =
p.producteur_id WHERE f.id = ?
```

Résultat attendu. http://localhost:3000/api/producteurs renvoie les 4 producteurs. `GET /api/producteurs/<id de la Ferme des Trois Chênes>/produits` renvoie ses 4 légumes.

Indice. Pour `GET /:id/produits`, appelez d'abord `depot.trouverParId(id)`. Sans cela, un producteur inconnu donnerait `[]` au lieu de 404. Dans le fichier de routes, la route `/:id/produits` se déclare comme les autres : `router.get('/:id/produits', ...)`.

Pour aller plus loin : supprimez un producteur qui a encore des produits. Avec MySQL, la clé étrangère refuse et l'API répond 409. Avec Mongo, rien ne l'empêche. Ajoutez la vérification dans `supprimer` avec `Produit.countDocuments({ producteurId: id })`.

À rendre. Les quatre fichiers ci-dessus, et `app.js`.

Exercice 4 (souhaitable) : passer une commande

Ce qui est demandé. Écrivez `passerCommande` et `afficherCommande` dans `controllers/commande.controller.js`, puis `creer` et `trouverParId` dans `depot/commandes.js`. Les routes sont déjà branchées : `POST /api/commandes` et `GET /api/commandes/:id`.

Le client envoie :

```
{ "client": "Léa Martin", "lignes": [ { "produitId": 1, "quantite": 2 }, { "produitId": 12, "quantite": 1 } ] }
```

Ces id sont ceux de la version MySQL. Avec Mongo, prenez les id des Pommes Reinette et du Miel d'acacia dans `GET /api/produits`, entre guillemets.

Les règles GreenCart.

1. Le client est obligatoire. Il faut au moins une ligne.
2. Chaque quantité est un nombre entier, au moins 1, au plus le stock du produit.
3. On vérifie **toutes** les lignes avant de retirer le moindre stock. Une commande refusée ne change aucun stock.
4. Le stock ne devient jamais négatif.
5. La règle de calcul est celle de l'atelier 1, avec les mêmes noms de fonctions :
   - total HT = somme des prix HT x quantité, **sans arrondir** ligne par ligne
   - total TTC = `arrondirAuCentime(totalHT x 1,055)`, c'est `prixTTC(totalHT)`
   - livraison = 4,90 € si le total TTC est sous 40 €, sinon 0 (et 0 pour un total nul), c'est `fraisLivraison(totalTTC)`
   - montant à payer = `arrondirAuCentime(totalTTC + livraison)`
6. `arrondirAuCentime` est fournie. Reprenez `prixTTC` et `fraisLivraison` de l'atelier 1.

Les réponses :

Cas	Code
Commande enregistrée	201, avec la commande
Client manquant, aucune ligne, quantité invalide	400
Produit inconnu	404
Stock insuffisant	409 (conflit)

Rappel. Cours Node.js et Express, slide 20 (codes de statut), slide 31 (`try`/`catch`), slide 32 (middleware d'erreur). Pour la TVA et la livraison, vous avez déjà écrit ces calculs à l'atelier 1.

- Mongo : `$inc` ajoute une valeur à un champ. Avec une valeur négative, il retire : `Produit.findByIdAndUpdate(id, { $inc: { stock: -2 } })`. Puis `Commande.create({ ... })`. Le modèle est dans `models/commande.model.js`.
- MySQL : trois requêtes.

```
UPDATE produits SET stock = stock - ? WHERE id = ? AND stock >= ?
INSERT INTO commandes (client, total_ht, total_ttc, frais_livraison, montant_a_payer) VALUES (?, ?, ?, ?, ?)
INSERT INTO lignes_commande (commande_id, produit_id, quantite, prix_unitaire_ht) VALUES (?, ?, ?, ?)
```

`AND stock >= ?` protège le stock : s'il ne suffit pas, aucune ligne n'est modifiée et `affectedRows` vaut 0.

Résultat attendu. 2 Pommes Reinette (3,20 €) et 1 Miel d'acacia (10,50 €) :

```
{
  "id": 1,
  "client": "Léa Martin",
  "statut": "en_attente",
  "lignes": [
    { "produitId": 1, "quantite": 2, "prixUnitaireHT": 3.2 },
    { "produitId": 12, "quantite": 1, "prixUnitaireHT": 10.5 }
  ],
  "totalHT": 16.9,
  "totalTTC": 17.83,
  "fraisLivraison": 4.9,
  "montantAPayer": 22.73
}
```

Le stock des Pommes Reinette passe de 40 à 38. 2 Tommes de chèvre (22 €) : total HT 44, total TTC 46,42, livraison 0, montant à payer 46,42. 2 Pommes Reinette et 3 Miels d'acacia : total HT 37,90, total TTC 39,98, livraison 4,90, montant à payer 44,88. Si vous trouvez 40,00 et une livraison offerte, vous avez arrondi chaque ligne : c'est le piège de l'atelier 1. 5 Radis alors qu'il en reste 3 : `409` et un message qui dit le stock disponible.

Les noms des champs `totalHT`, `totalTTC`, `fraisLivraison` et `montantAPayer` sont imposés : le vérificateur les lit. Dans le contrôleur, ne nommez pas une variable `fraisLivraison` : ce nom est déjà pris par la fonction. Le corrigé l'appelle `livraison`.

Indice. Le contrôle du corps (étape 0) peut aussi s'écrire dans un middleware, `middlewares/validation-commande.middleware.js`, sur le modèle de `validation.middleware.js`. Le corrigé fait ainsi. Le squelette du contrôleur donne les étapes dans l'ordre. Écrivez-les une par une et testez entre chaque. Pour la quantité, `Number.isInteger(quantite) && quantite >= 1` refuse `0`, `1.5`, `-2` et `"2"`. Gardez le prix du jour dans chaque ligne (`prixUnitaireHT`) : si le prix change demain, la commande reste juste. Pour répondre 409 depuis le dépôt : `const err = new Error('Stock insuffisant'); err.status = 409; throw err;`. Le middleware d'erreur utilise `err.status`.

Pour aller plus loin (MySQL) : une transaction. Si le serveur s'arrête entre le `UPDATE` et le `INSERT`, le stock est retiré sans commande. Une transaction regroupe les requêtes : soit toutes sont faites, soit aucune. C'est la bonne façon de faire.

```
const connexion = await pool.getConnection();   // une transaction se fait sur UNE connexion
try {
  await connexion.beginTransaction();
  // ... vos requêtes, avec connexion.execute(...) au lieu de pool.execute(...)
  await connexion.commit();                     // tout est bon : on valide
} catch (err) {
  await connexion.rollback();                   // un problème : on annule tout
  throw err;
} finally {
  connexion.release();                          // on rend la connexion au pool
}
```

Pour aller plus loin (Mongo). Mettez la condition de stock dans le filtre : `Produit.findOneAndUpdate({ _id: id, stock: { $gte: quantite } }, { $inc: { stock: -quantite } })`. Si le stock manque, la fonction renvoie `null` et rien n'est retiré.

À rendre. `controllers/commande.controller.js` et `depot/commandes.js`.

Exercice 5 (bonus) : connexion avec JWT

Le cours cite le JWT sans le détailler (slide 28). Ce bonus est donc très guidé.

Le principe. L'utilisateur envoie son email et son mot de passe. Le serveur vérifie, puis renvoie un **jeton** signé. Ensuite, le client joint ce jeton à chaque requête, dans l'en-tête `Authorization: Bearer <jeton>`. Un middleware vérifie la signature du jeton. Sans jeton valide : 401. Le mot de passe n'est jamais stocké en clair. La base garde une empreinte faite par `bcrypt`.

Compte de test, déjà dans la base : `admin@greencart.test`, mot de passe `greencart`. `bcryptjs` et `jsonwebtoken` sont déjà installés.

Ce qui est demandé.

1. `depot/utilisateurs.js` : écrivez `trouverParEmail(email)`. Il renvoie `{ id, email, motDePasseHash, role }` ou `null`.
2. `controllers/auth.controller.js` : suivez les 4 étapes écrites en commentaire. Le code de chaque étape est donné.
3. `middlewares/auth.middleware.js` : complétez les 3 lignes marquées `À compléter`.
4. `routes/produit.routes.js` : protégez POST, PUT et DELETE en plaçant `verifierToken` avant les autres middlewares.

```
router.post('/', verifierToken, validerProduit, controleur.creerProduit);
```

Rappel. Cours Node.js et Express, slide 28 (le middleware d'authentification vérifie le token JWT), slide 29 (middleware local, sur une seule route).

Résultat attendu.

```
curl -X POST http://localhost:3000/api/auth/connexion \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@greencart.test","motDePasse":"greencart"}'
# {"token":"eyJhbGciOi..."}

curl -i -X DELETE http://localhost:3000/api/produits/<id>
# 401 {"message":"Connexion requise"}

curl -i -X DELETE http://localhost:3000/api/produits/<id> -H "Authorization: Bearer
eyJhbGciOi..."
# 204
```

Un mauvais mot de passe donne `401`. Les GET restent ouverts à tous.

Indice. Un middleware doit **toujours** finir par `next()` ou par une réponse. Sinon la requête reste bloquée. Le vérificateur le signale : « le serveur n'a pas répondu en 8 secondes ». Une fois les routes protégées, le vérificateur se connecte tout seul avec le compte de test.

À rendre. `depot/utilisateurs.js`, `controllers/auth.controller.js`, `middlewares/auth.middleware.js`, `routes/produit.routes.js`.

Rendre votre travail

Rendez le dossier de votre version (`mongo/` ou `mysql/`) selon la consigne du formateur. Ne rendez jamais `node_modules/` ni `.env`. Le fichier `.gitignore` les exclut déjà. Joignez la sortie de `node ../verifier.js`.

Si vous êtes bloqué

- Le serveur ne démarre pas : lisez la première ligne d'erreur dans le terminal. Elle donne le fichier et la ligne.
- `Route introuvable` : le fichier de routes n'est pas branché dans `app.js`, ou le chemin est mal écrit.
- Réponse 500 : l'erreur complète s'affiche dans le terminal du serveur.
- La base ne répond pas : Docker Desktop est-il ouvert ? `docker compose ps` doit montrer le conteneur.
- Port déjà utilisé : changez `PORT` dans `.env`. Le vérificateur le lit.